Bloque 1: Control de Flujo, Operadores y Bucles

1. ¿Cuál es la diferencia principal entre la estructura tradicional `switch` y la expresión `match` (introducida en PHP 8)?

a) `switch` es más rápido y `match` no permite usar valores booleanos.

b) `match` realiza una comparación estricta (`===`) de tipos y valores, y retorna un valor directamente, mientras que `switch` utiliza comparación débil (`==`) y ejecuta bloques de código.

c) `match` solo funciona con cadenas de texto y `switch` solo con números enteros. d) No hay diferencias funcionales, `match` es solo un alias moderno para `switch`.

2. ¿Qué evalúa y devuelve el operador de fusión de null (`??`) en PHP? a)

Comprueba si una variable es igual a cero y devuelve verdadero o falso.

b) Retorna el primer operando si existe y no es nulo (`null`), de lo contrario retorna el segundo.

c) Elimina una variable de la memoria si su valor es nulo.

d) Compara de forma estricta dos valores nulos en una base de datos.

3. Dentro de los operadores lógicos en PHP, ¿qué diferencia existe entre `&&` y el operador `and`?

a) No hay ninguna diferencia en su comportamiento operativo.

b) `&&` solo funciona con cadenas y `and` con números.

c) Tienen distinta precedencia de operadores (prioridad de evaluación en expresiones complejas).

d) `and` está obsoleto desde PHP 7 y ya no se debe utilizar.

4. ¿Cuál es la característica fundamental que diferencia a un bucle `do-while` de un bucle `while` convencional?

a) El bucle `do-while` no puede utilizar la sentencia `break`.

b) El bloque de código del `do-while` se ejecuta al menos una vez antes de evaluar la condición.

c) El bucle `while` se utiliza exclusivamente para arreglos asociativos. d) El bucle `do-while` es más rápido porque omite la comprobación de variables.

5. ¿Para qué tipo de situaciones está diseñado principalmente el bucle `foreach` en comparación con el bucle `for` tradicional?

a) Para contar iteraciones numéricas estrictas usando un contador incremental.

b) Para recorrer y manipular de forma sencilla los elementos de un array (índices o claves y valores).

c) Para conectar bases de datos y realizar consultas SQL

repetitivas. d) Para leer archivos binarios de gran tamaño en el

servidor.

6. ¿Qué efecto producen las sentencias `break` y `continue` respectivamente dentro de un bucle?

a) `break` detiene y sale completamente del bucle; `continue` salta la iteración actual y pasa a la siguiente.

b) `break` reinicia el bucle desde el principio; `continue` finaliza el script por

completo. c) Ambas hacen exactamente lo mismo, interrumpir la ejecución del bucle.

d) `break` pausa el script 5 segundos y `continue` lo reanuda.

Bloque 2: Modularización y Funciones

7. ¿Qué diferencia principal existe entre `include` y `require` al intentar incluir un archivo que no existe?

a) `include` detiene la ejecución con un Fatal Error, mientras que `require` solo emite una advertencia (Warning) y el script continúa.

b) `include` emite una advertencia (Warning) y permite que el script siga ejecutándose, mientras que `require` genera un error fatal (Fatal Error) y detiene el script.

c) `require` solo funciona dentro de funciones y `include` en el ámbito

global. d) No hay diferencias técnicas, son sinónimos exactos.

8. ¿Cuál es el propósito de utilizar las variantes `include_once` o

`require_once`? a) Optimizar la velocidad del servidor comprimiendo el archivo

incluido.

b) Evitar que un archivo sea incluido múltiples veces por error, previniendo conflictos de redeclaración de funciones o clases.

c) Asegurar que el archivo solo se incluya si el usuario ha iniciado sesión. d)

Permitir la inclusión de archivos ubicados en servidores remotos externos.

9. Al crear una función propia en PHP con parámetros, ¿cómo se puede definir un valor por defecto para un parámetro?

a) Colocando un signo de exclamación antes del nombre: `function miFuncion(!$param = 10)`

b) Asignándole un valor directamente en la firma de la función utilizando el operador de asignación: `function miFuncion($param = "valor_por_defecto")`

c) Declarándolo obligatoriamente como una variable global dentro del

script. d) No se pueden asignar valores por defecto a los parámetros en PHP.

10. Si defines una función en un archivo llamado `funciones.php`, ¿cuál es el procedimiento correcto para utilizarla en tu archivo `index.php`?

a) Utilizar `include 'funciones.php';` o `require 'funciones.php';` al inicio de `index.php`, y luego llamar a la función normalmente por su nombre.

b) Escribir el nombre del archivo seguido de un punto y la función: `funciones.php.miFuncion()`

c) Declarar la función nuevamente de manera obligatoria en `index.php`. d)

Utilizar la etiqueta HTML `<script src="funciones.php"></script>`.

Bloque 3: Fechas, Seguridad y Formularios

11. ¿Qué función de PHP se debe utilizar obligatoriamente antes de invocar a `date()` para evitar discrepancias en la hora devuelta por el servidor?

a)`set_time_limit()`

b)`date_default_timezone_set()`

do)`time_zone_local()`

d)`php_ini_time()`

12. ¿Qué devuelve la función nativa de PHP `time()`?

a) La hora exacta en formato de 12 horas con AM/PM en formato de texto.

b) La cantidad de segundos transcurridos desde el Época Unix (1 de enero de 1970).

c) Un arreglo asociativo con el día, mes, año, hora, minuto y segundo actual.

d) El tiempo estimado de ejecución del script actual en milisegundos.

13. ¿Por qué razón los algoritmos antiguos como `md5()` o `sha1()` ya no se consideran seguros para almacenar contraseñas de usuarios?

a) Porque consumen demasiada memoria RAM del servidor web.

b) Porque son algoritmos rápidos de calcular, lo que facilita enormemente ataques de fuerza bruta y uso de tablas precalculadas (rainbow tables).

c) Porque PHP eliminó su soporte de forma definitiva en las versiones modernas.

d) Porque generan hashes demasiado largos imposibles de almacenar en bases de datos relacionales.

14. ¿Cuál es el propósito fundamental de la función `password_hash()` en PHP?

a) Cifrar datos sensibles de una tarjeta de crédito para poder descifrarlos y leerlos más tarde.

b) Generar un hash seguro unidireccional utilizando algoritmos fuertes (como Bcrypt o Argon2) e incorporando sal (salt) de forma automática.

c) Comprobar si una contraseña cumple con tener números, mayúsculas y símbolos.

d) Encriptar la contraseña mediante un algoritmo reversible basado en clave secreta.

15. ¿Cómo se comprueba de manera segura si una contraseña ingresada por un usuario coincide con el hash almacenado en la base de datos?

a) Comparando ambas cadenas directamente con el operador de igualdad estricta

(`===`). b) Utilizando la función `md5($password_ingresado) == $hash_guardado`.

c) Mediante la función `password_verify($password_ingresado, $hash_guardado)`.

d) Usando la función `password_decrypt()`.

16. En relación al envío de datos mediante formularios HTML, ¿cuál es la principal diferencia entre los métodos `GET` y `POST`?

a) `GET` envía los datos ocultos en el cuerpo de la petición HTTP y `POST` los muestra en la URL.

b) `GET` expone los datos en la barra de direcciones (URL) con límite de tamaño, mientras que `POST` los envía de manera oculta y sin límite estricto de tamaño en el cuerpo de la petición.

c) `GET` solo funciona con archivos locales y `POST` con servidores web

remotos. d) No hay diferencias de seguridad, ambos métodos ocultan los datos

por igual.

17. Al procesar un formulario que contiene un menú desplegable (`<select name="pais">`), ¿en qué variable superglobal de PHP se recibe la opción seleccionada si se utilizó el método `POST`?

a)`$_GET['pais']`

b)`$_FILES['pais']`

do)`$_POST['pais']`

d)`$_SERVER['pais']`

18. ¿Cómo se deben definir en el código HTML las casillas de verificación (checkboxes) múltiples para que PHP las reciba correctamente como un arreglo de datos?

a)`name="intereses"`

b)`name="intereses[]"`

do)`name="array_intereses"`

d)`name="intereses_multiple"`

19. Cuando un usuario marca varias casillas de verificación configuradas como un array en un formulario `POST`, ¿cómo se accede a dichos valores en PHP?

a) A través de un bucle (como `foreach`) recorriendo la variable `$_POST['nombre_del_checkbox']`.

b) Utilizando únicamente la función `$_POST['nombre_del_checkbox'][0]` para obtener todo junto.

c) Convirtiendo el formulario obligatoriamente a formato JSON mediante JavaScript.

d) No se pueden enviar múltiples checkboxes en PHP sin usar bases de datos previas.

20. Si deseas verificar si un formulario con método `POST` ha sido enviado antes de procesar sus datos, ¿qué combinación de funciones y variables es la más recomendada?

hacia`if (isset($_POST['submit']))`

a)`if ($_SERVER['REQUEST_METHOD'] == 'POST')`

b)`if (file_exists($_POST))`

do)`if (check_form() == true)`

d)`if (empty($_POST))`